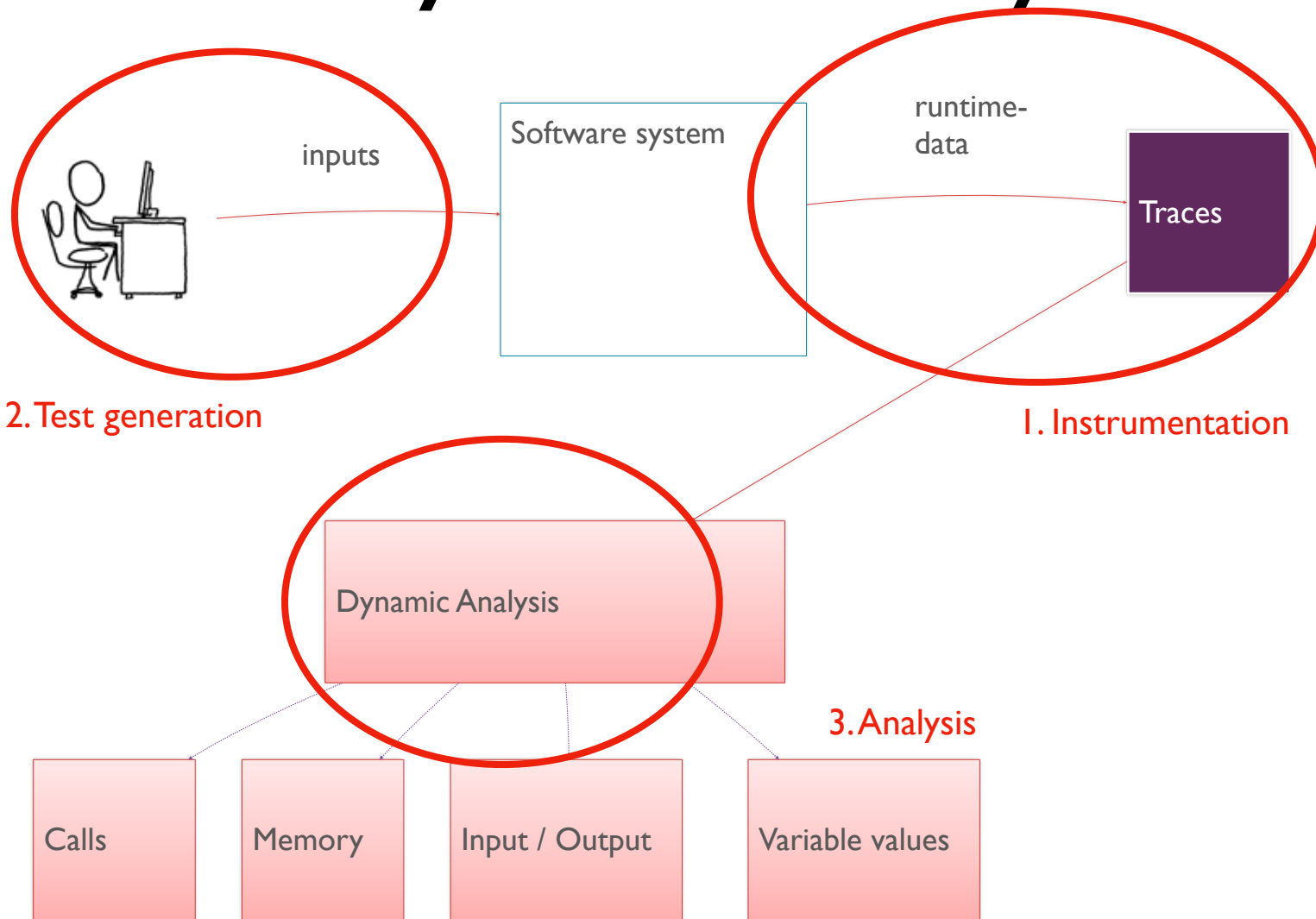


Software Analysis

(Dynamic) Symbolic Execution

Gordon Fraser
Lehrstuhl für Software Engineering II

Dynamic Analysis



Test Generation

- Symbolic Execution
- Dynamic Symbolic Execution
Start with random values
- Random Testing
Test only random values
- Fuzzing
Generate random values by mutating inputs
- Search-based Testing
Improve random values using evolutionary search
- Machine-Learning Based Testing
Query LLMs / generative models

Symbolic Execution

Example

```
1. void foobar(int a, int b) {  
2.   int x = 1, y = 0;  
3.   if (a != 0) {  
4.     y = 3 + x;  
5.     if (b == 0)  
6.       x = 2 * (a + b);  
7.   }  
8.   assert (x - y != 0);  
9. }
```

Which inputs make the assert at line 8 fail?

Symbolic State

- At any point during program execution, symbolic execution keeps two formulas:
 - **Symbolic store σ :** *A symbolic memory store σ that maps variables to symbolic expressions or values*
 - **Path constraint π :** *A first-order Boolean formula that describes the conditions satisfied by the branches taken along the path*
- Therefore, at any point in time the symbolic state is described as the **conjunction ($\wedge$)** of these two formulae

Symbolic Store

- The values of variables at any moment in time are given by a function $\sigma \in \text{SymStore} = \text{Var} \mapsto \text{Sym}$
- Arithmetic expressions manipulate symbolic values
- During execution, **assignments** affect the symbolic store (but not the path constraint)

Symbolic Store

- Let $\sigma : x \mapsto x_0, y \mapsto y_0$
- Lets evaluate statement 6: $z = x + y$
- After executing statement 6, the symbolic store becomes $\sigma : x \mapsto x_0, y \mapsto y_0, z \mapsto x_0 + y_0$

Path Constraint

- Records the history of all branches taken so far in a formula (π)
- The formula is typically in a decidable logical fragment without quantifiers
- At the start of the analysis π is *true*
- During execution, the **evaluation of conditionals** affects the path constraint (but not the symbolic store)

Path Constraint

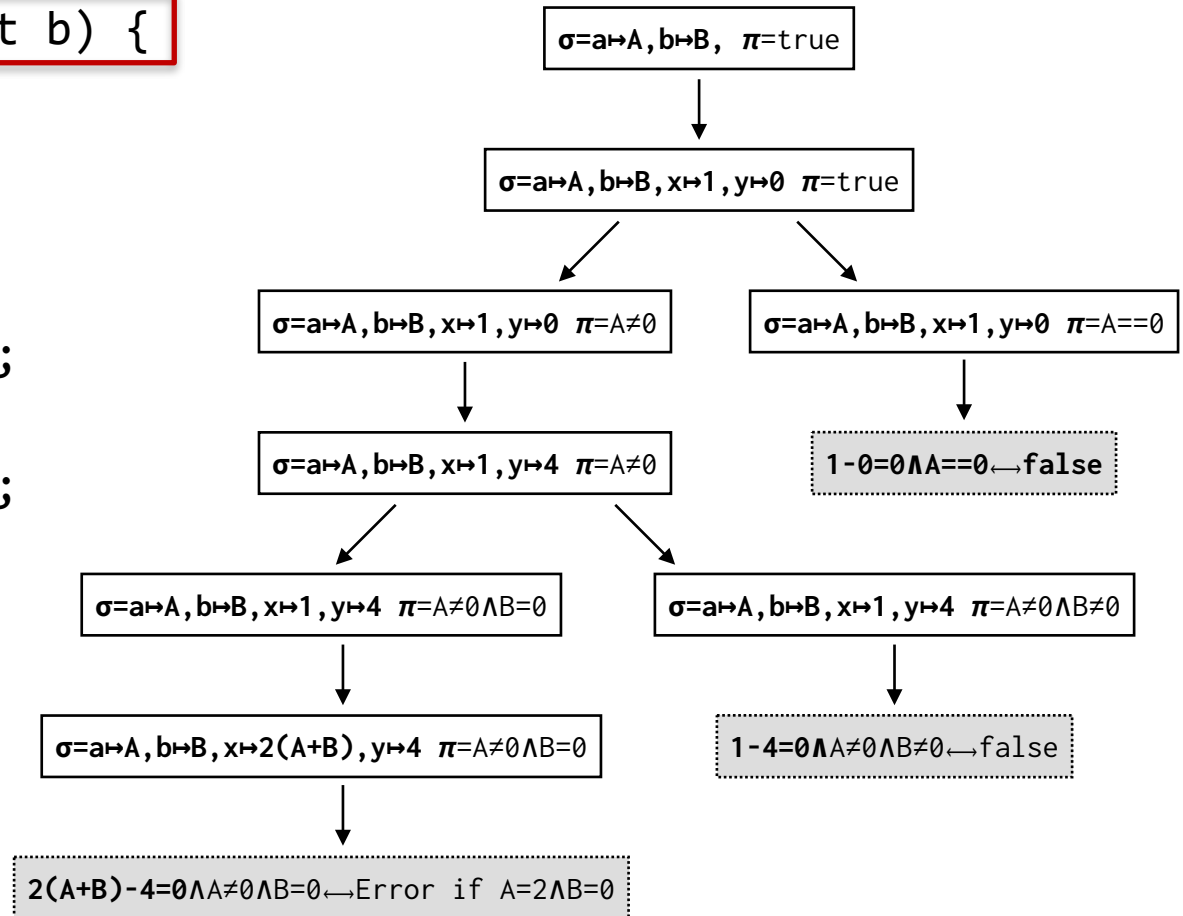
- Let $\sigma : x \mapsto x_0, y \mapsto y_0, \quad \pi = x_0 > 10$
- Lets evaluate: `if (x > y + 1) {5: ... }`
- At label 5, symbolic store σ does not change,
but the path constraint becomes:
$$\pi = x_0 > 10 \wedge x_0 > y_0 + 1$$

Example

```

1. void foobar(int a, int b) {
2.   int x = 1, y = 0;
3.   if (a != 0) {
4.     y = 3 + x;
5.     if (b == 0)
6.       x = 2 * (a + b);
7.   }
8.   assert (x - y != 0);
9. }

```



Implicit Checks

- Implicit checks before each dangerous operation *fork* the execution (errors lurk in those paths!)
 - Division/modulo: one branch in which divisor is 0 and another where it isn't
 - Pointer dereferences: null vs not-null
 - Array indexing: negative vs positive index
 - Assert statements: true vs false

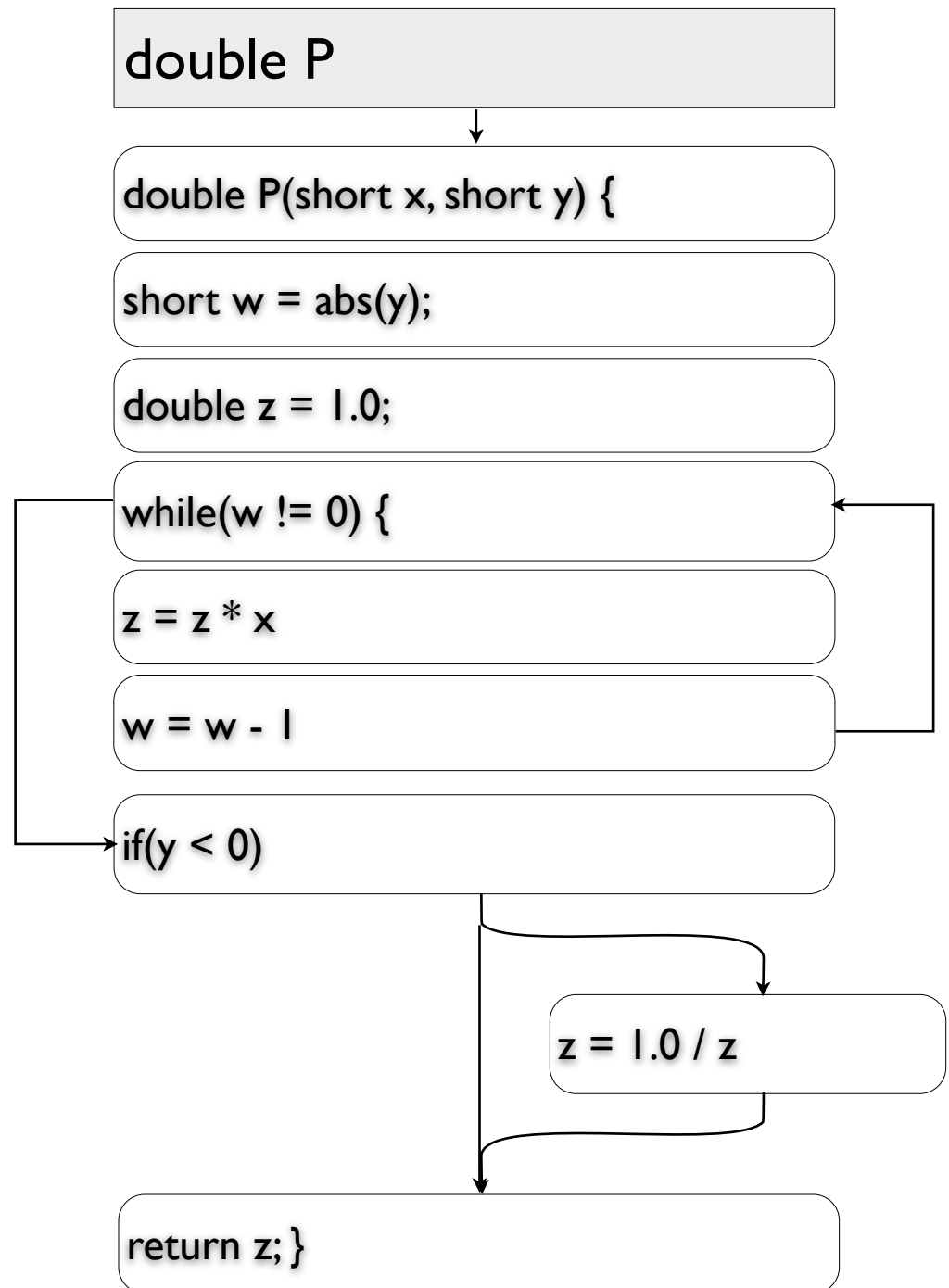
Test Generation

- Path constraints represent all concrete inputs that induce specific execution paths
- Solving a path constraint yields input values that exercise the corresponding execution path
- Can generate test suites that cover a given test criterion (all branches)
- Algorithm:
 1. Select path
 2. Generate the path condition
 3. Solve the path condition to generate test data

Example

```
double P(short x, short y) {  
    short w = abs(y);  
    double z = 1.0;  
    while(w != 0) {  
        z = z * x;  
        w = w - 1;  
    }  
    if(y < 0)  
        z = 1.0 / z;  
  
    return z;  
}
```

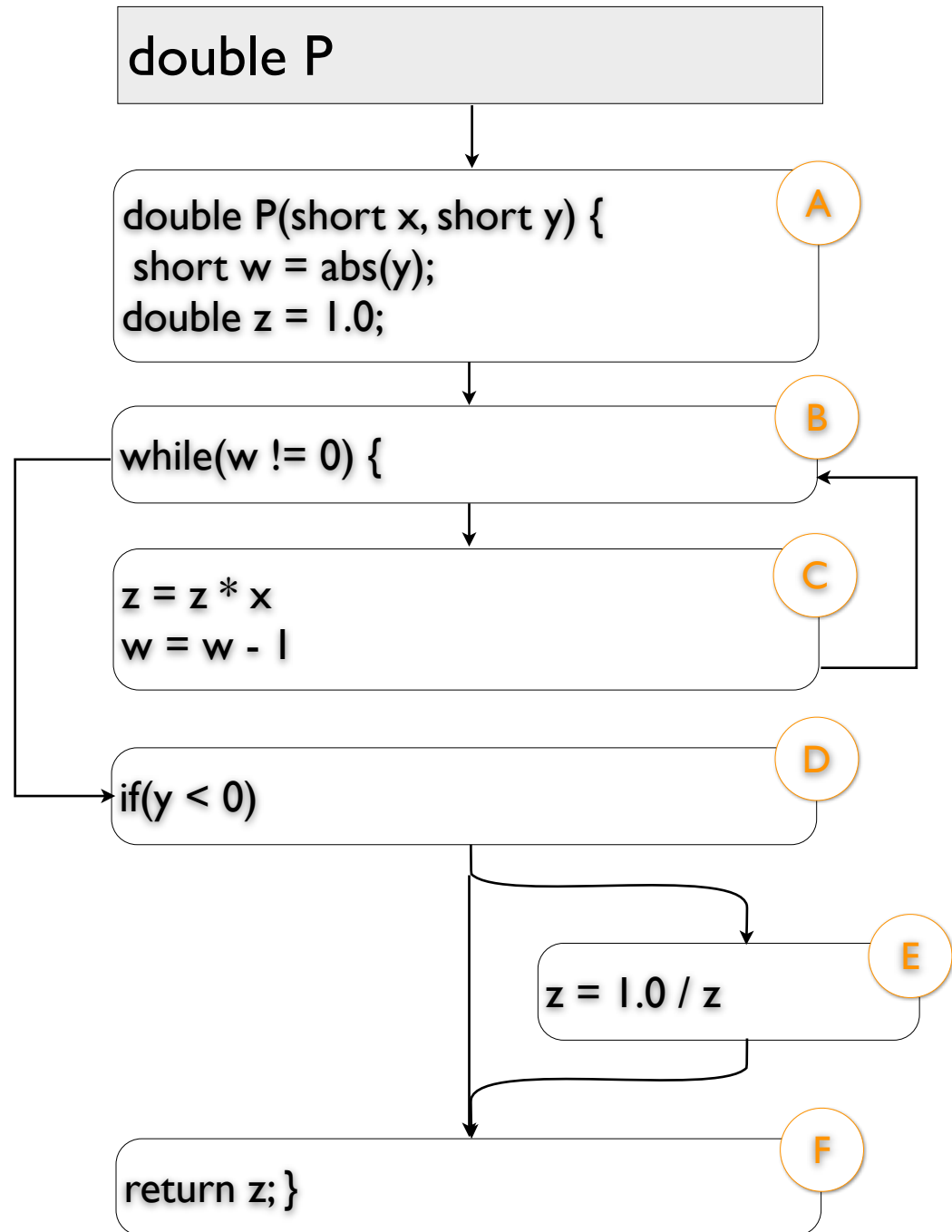
```
double P(short x, short y) {  
    short w = abs(y);  
    double z = 1.0;  
    while(w != 0) {  
        z = z * x;  
        w = w - 1;  
    }  
    if(y < 0)  
        z = 1.0 / z;  
  
    return z;  
}
```



```

A double P(short x, short y) {
B   short w = abs(y);
C   double z = 1.0;
    while(w != 0) {
        z = z * x;
        w = w - 1;
    }
D   if(y < 0)
E       z = 1.0 / z;
F   return z;
}

```



Goal: A-B-C-B-C-B-D-F with $x \mapsto X, y \mapsto Y$

A

$\sigma = w \mapsto \text{abs}(Y), z \mapsto 1.0$

$\pi = \top$

A-B

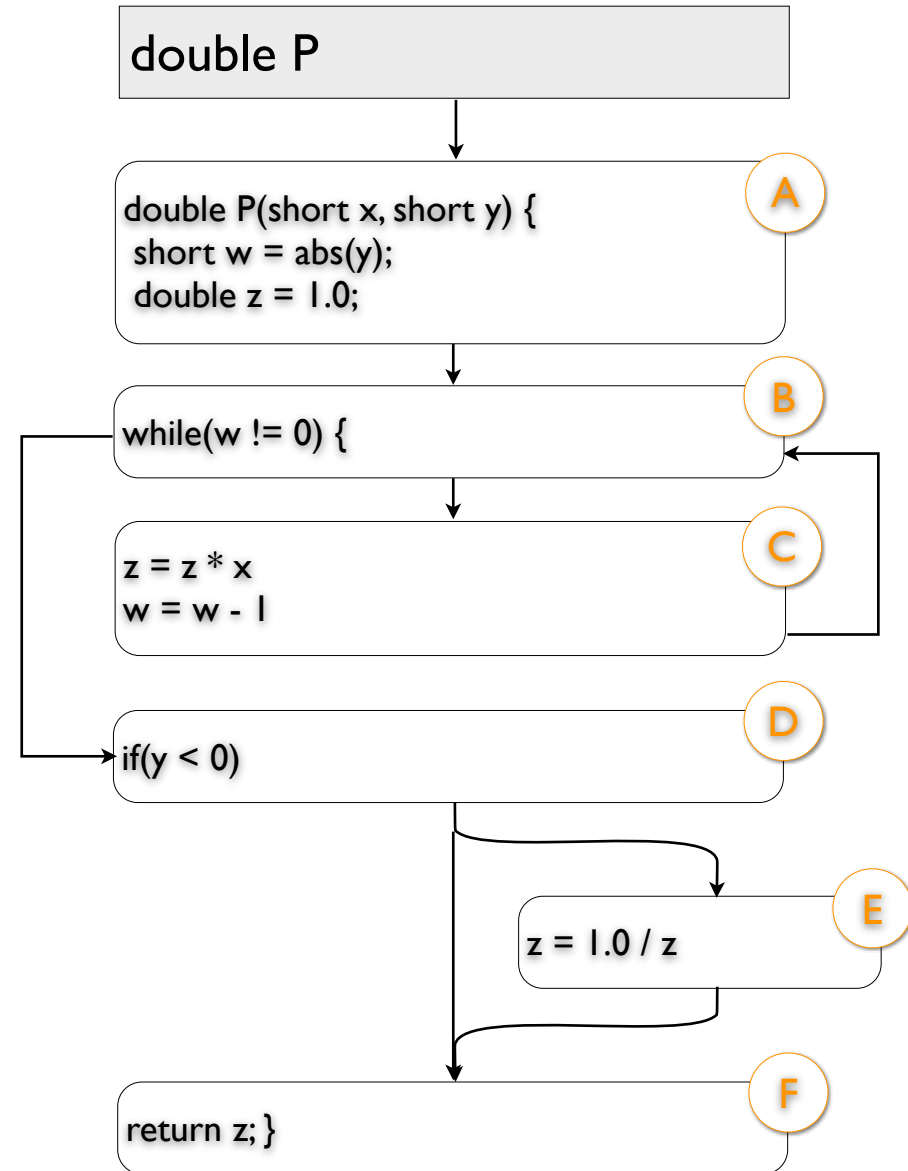
$\sigma = w \mapsto \text{abs}(Y), z \mapsto 1.0$

$\pi = \text{abs}(Y) \neq 0$

A-B-C:

$\sigma = z \mapsto X, w \mapsto \text{abs}(Y) - 1$

$\pi = \text{abs}(Y) \neq 0$



Goal: A-B-C-B-C-B-D-F with $x \mapsto X, y \mapsto Y$

A-B-C-B:

$\sigma = z \mapsto X, w \mapsto \text{abs}(Y) - 1;$

$\pi = \text{abs}(Y) \neq 0 \wedge \text{abs}(Y) - 1 \neq 0$

A-B-C-B-C:

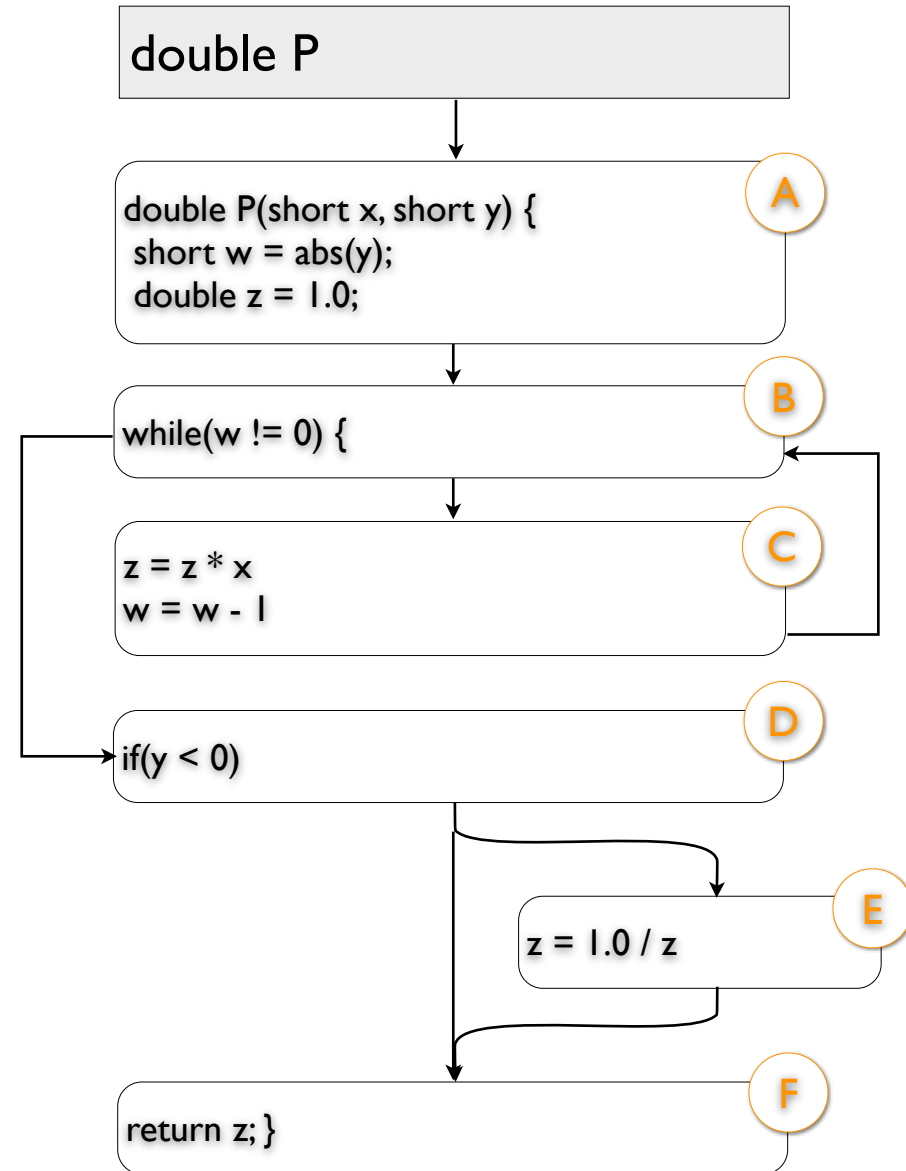
$\sigma = z \mapsto X * X, w \mapsto \text{abs}(Y) - 2;$

$\pi = \text{abs}(Y) \neq 0 \wedge \text{abs}(Y) - 1 \neq 0$

A-B-C-B-C-B:

$\sigma = z \mapsto X * X, w \mapsto \text{abs}(Y) - 2;$

$\pi = \text{abs}(Y) \neq 0 \wedge \text{abs}(Y) - 1 \neq 0 \wedge$
 $\text{abs}(Y) - 2 = 0$



Goal: A-B-C-B-C-B-D-F with $x \mapsto X, y \mapsto Y$

A-B-C-B-C-B-D:

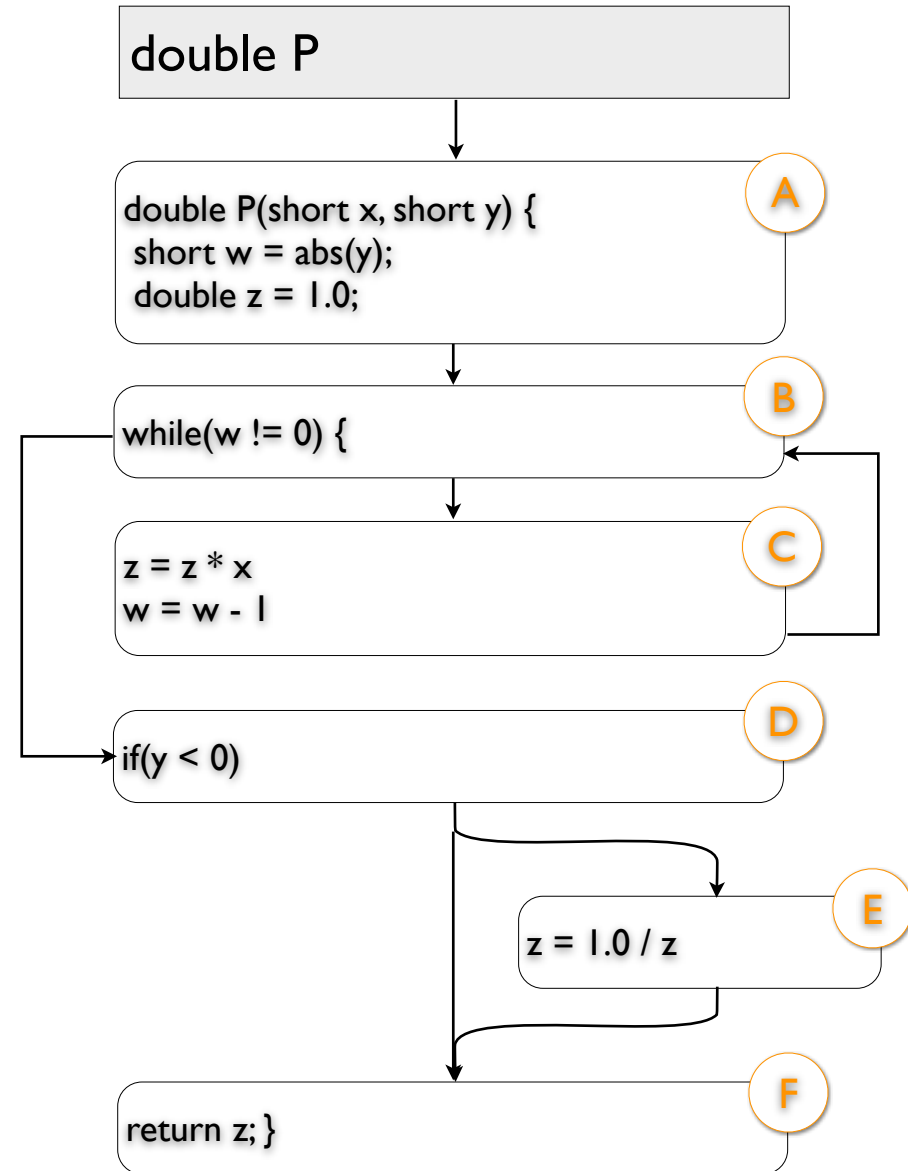
$\sigma = z \mapsto X * X, w \mapsto \text{abs}(Y) - 2;$

$\pi = \text{abs}(Y) \neq 0 \wedge \text{abs}(Y) - 1 \neq 0 \wedge$
 $\text{abs}(Y) - 2 = 0 \wedge Y \geq 0$

A-B-C-B-C-B-D-F:

$\sigma = z \mapsto X * X, w \mapsto \text{abs}(Y) - 2;$

$\pi = \text{abs}(Y) \neq 0 \wedge \text{abs}(Y) - 1 \neq 0 \wedge$
 $\text{abs}(Y) - 2 = 0 \wedge Y \geq 0$

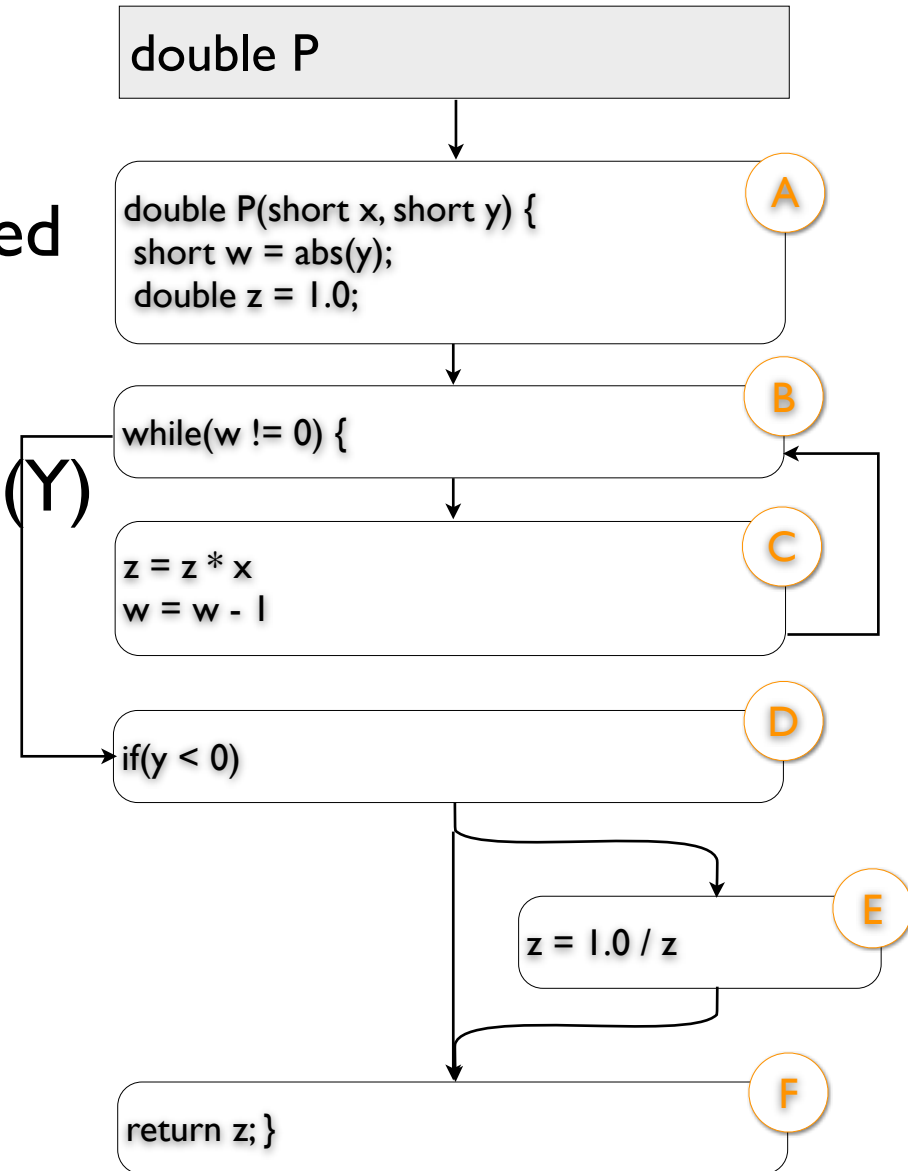


Goal: A-B-C-B-C-B-D-F with $x \mapsto X, y \mapsto Y$

To generate a test case that executes the target path we need to solve the constraint:

$$\text{abs}(Y) \neq 0 \wedge \text{abs}(Y) - 1 \neq 0 \wedge \text{abs}(Y) - 2 = 0 \wedge Y \geq 0$$

$$X=0, Y=2$$



Limitations of Symbolic Execution

Loops

```
int F(unsigned int k) {  
    int sum = 0;  
    int i = 0;  
    for ( ; i < k; i++)  
        sum += i;  
    return sum;  
}
```

- Unbounded loops make the symbolic execution keep running forever
- A common solution is to bound the loop (*under approximation*)
- Another solution is to *manually* provide a loop invariant (*over approximation*)

Constraint Solver

```
def fermat3(x,y,z):  
    if (x > 0 and y > 0 and z > 0):  
        if (x*x*x + y*y*y == z*z*z):  
            return "Fermat and Wiles were wrong!?!"  
    return 0
```

- Some problems *cannot* be solved others are simply too complex (e.g., Integers vs Strings)

Opaque Functions & Environmental Modeling

```
def example(x,y):  
    if (foo(x) == y):  
        return 1  
    return 0
```

```
def sum_input(x,y):  
    a = input()  
    s = x + y +  
    print("x+y")
```

- If the source code is not available, symbolic execution is not possible

More Limitations

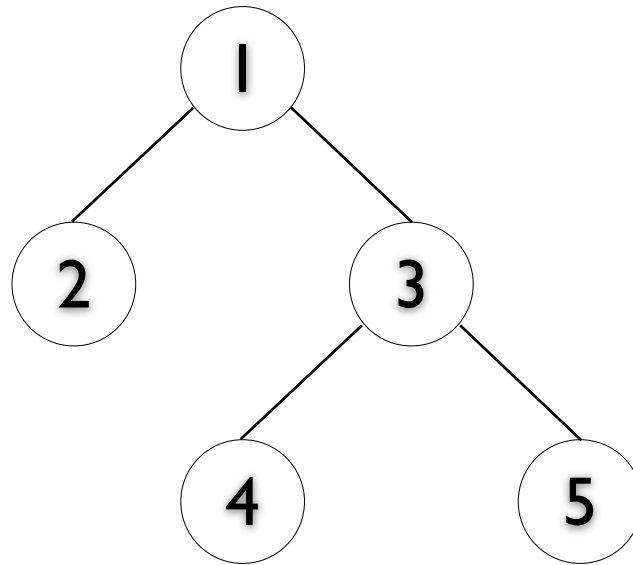
- Recursions --- infinite execution tree
- Path explosion --- exponentially many paths
- Heap modeling --- symbolic data structures and pointers
- Infeasible paths

Dynamic Symbolic Execution

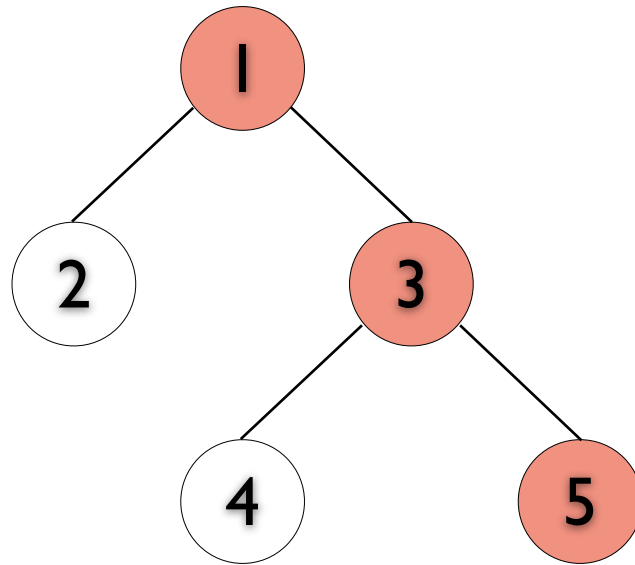
Dynamic Symbolic Execution

- Symbolic execution of a **concrete** execution
- Also called **concolic** execution
concrete+symbolic
- Uses (real) input values to select feasible paths
automatically
- Requires instrumenting each statement of P

Dynamic Symbolic Execution

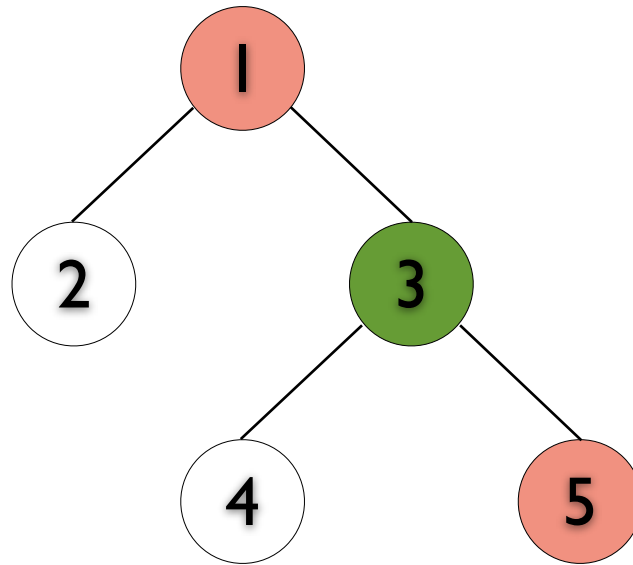


Dynamic Symbolic Execution



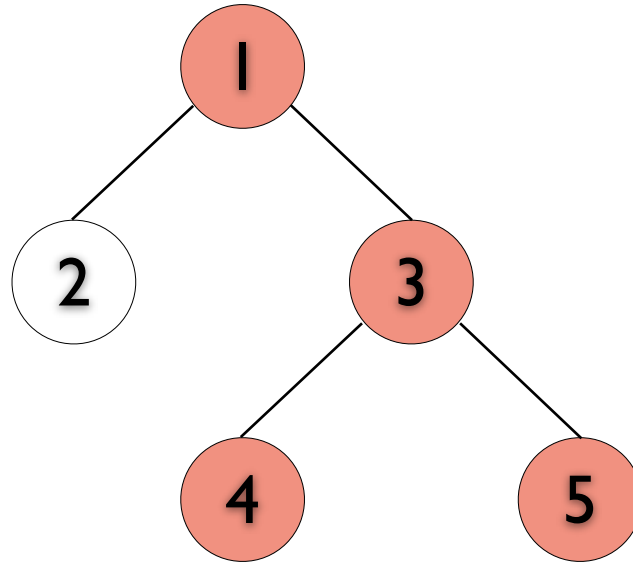
Generate a random input and execute the corresponding feasible path; collect the path condition during execution

Dynamic Symbolic Execution



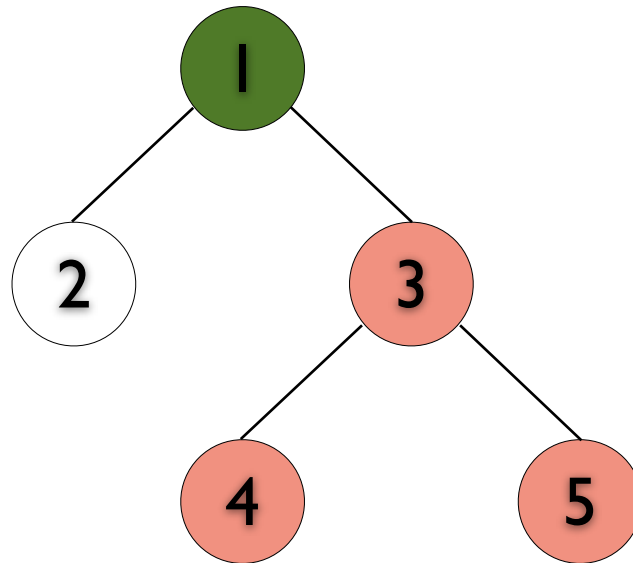
Try to solve the CS where the last constraint is refuted

Dynamic Symbolic Execution



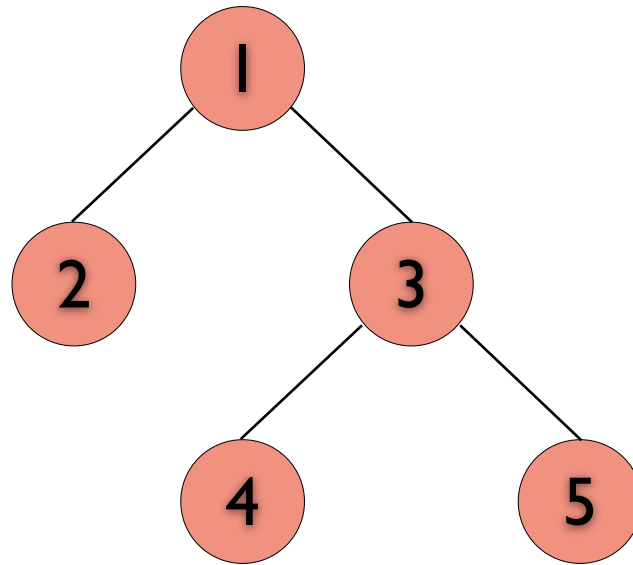
Execute the input data that solves the constraint

Dynamic Symbolic Execution



Try to solve the CS where the next constraint is refuted

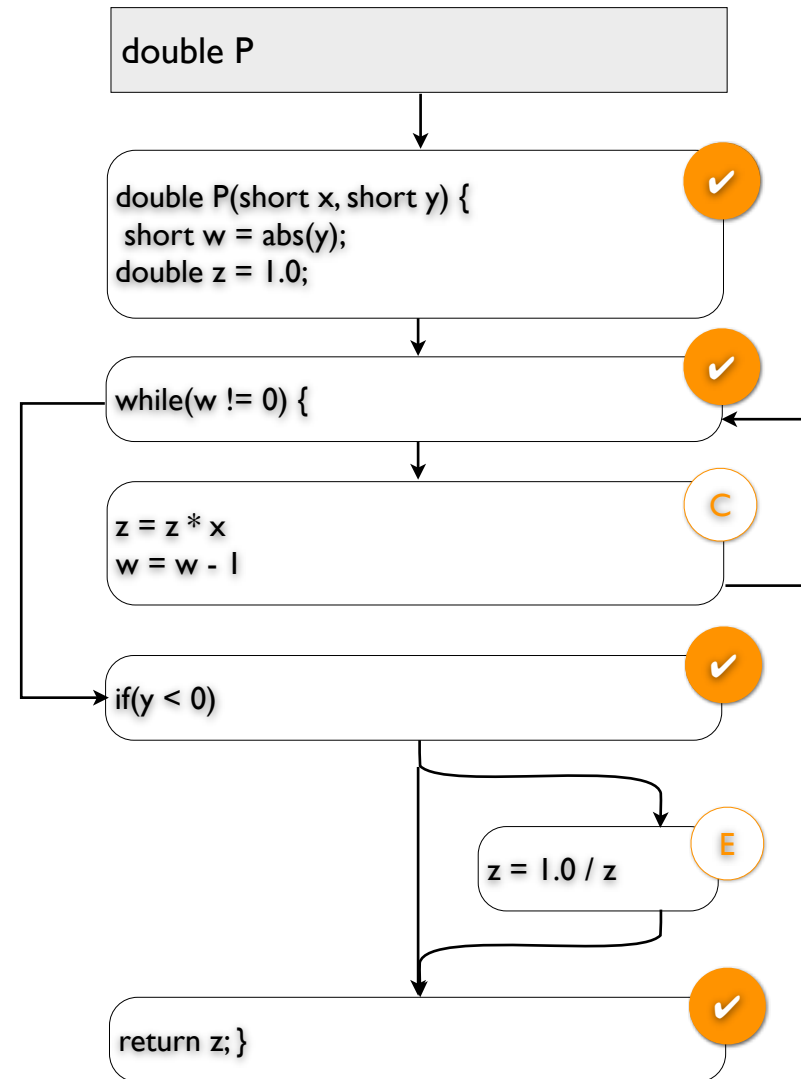
Dynamic Symbolic Execution



...until all paths have been covered

Generate a random input and
execute the corresponding
feasible path

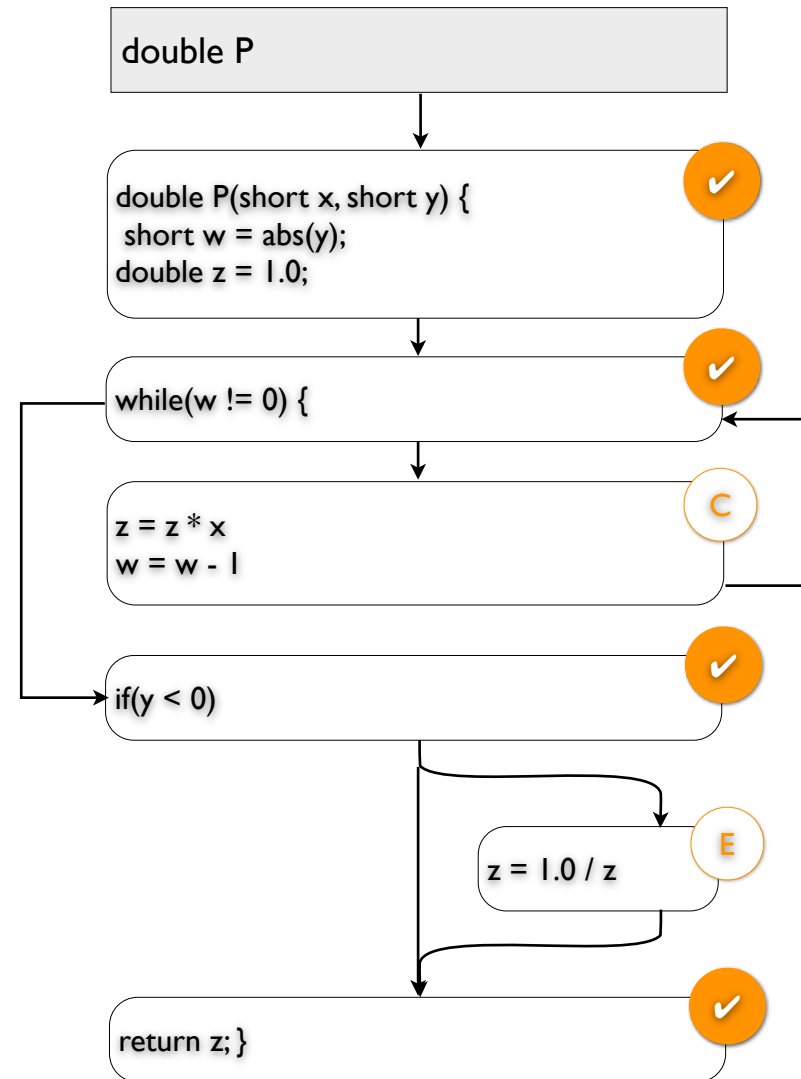
(0,0)



While executing, collect
constraints along path taken

(0,0)

$\text{abs}(Y) = 0$
 $\neg(Y < 0)$

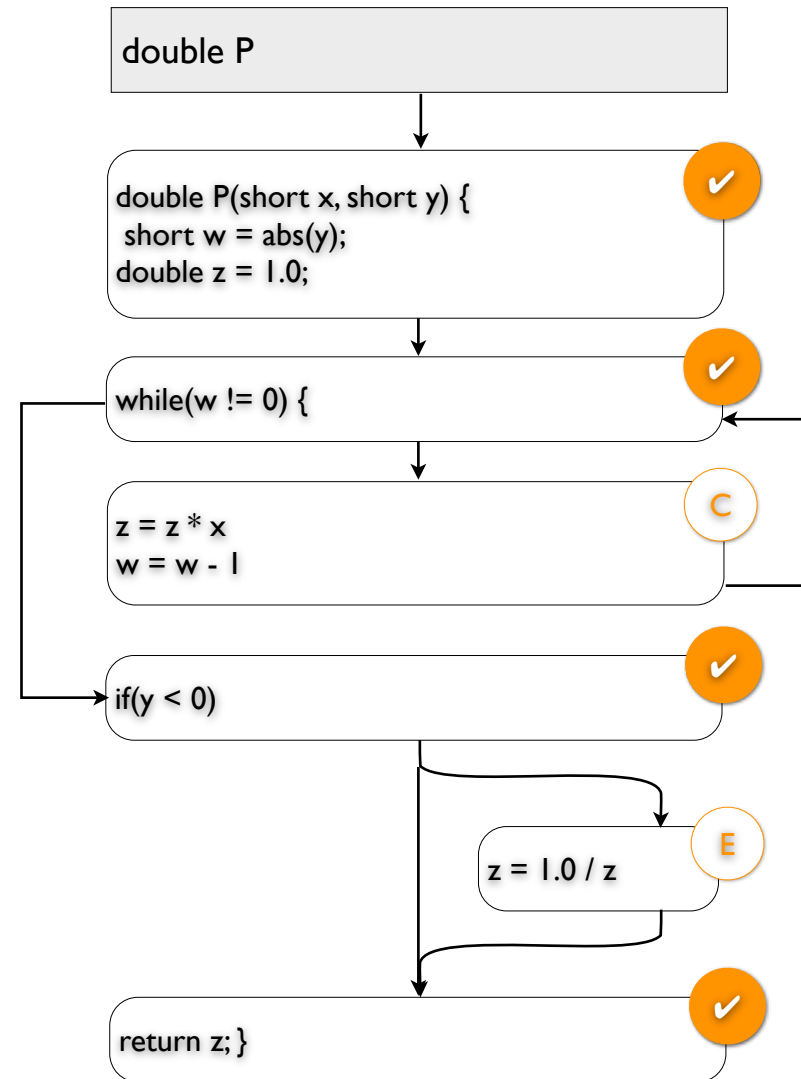


Try to solve the CS where the last constraint is refuted

(0,0)

$\text{abs}(Y) = 0$
($Y < 0$)

Unsatisfiable constraint

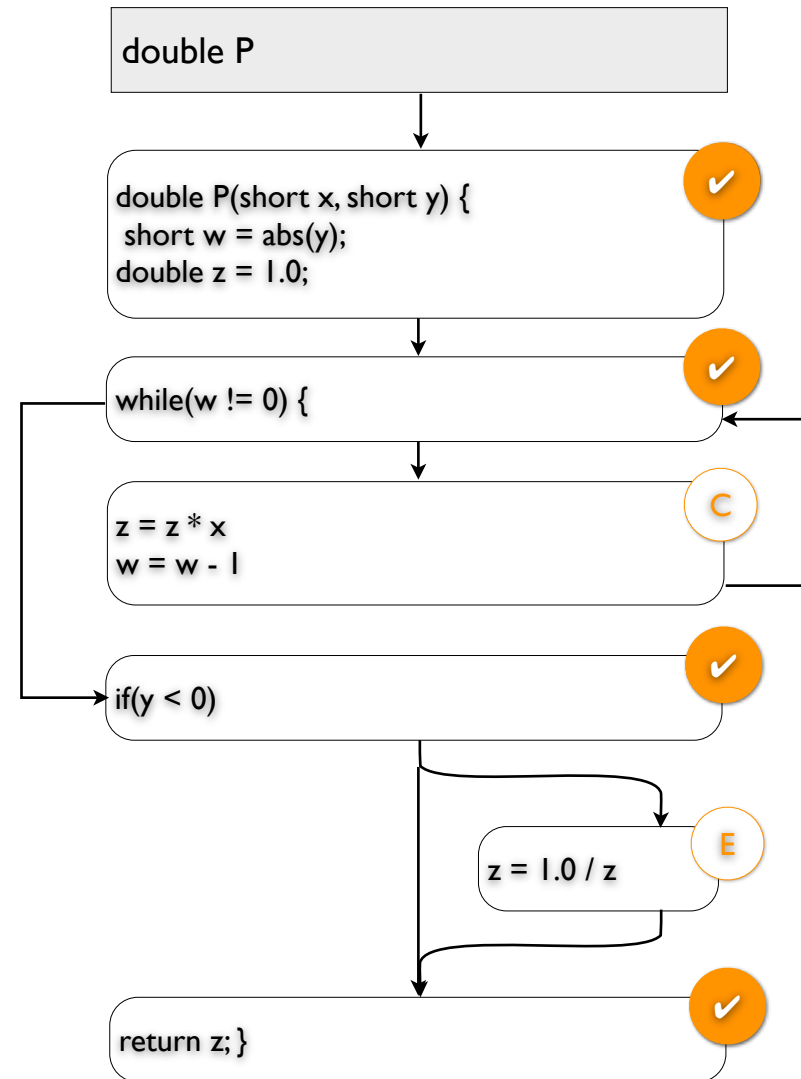


Try to solve the CS where the last constraint is refuted

(0,0)

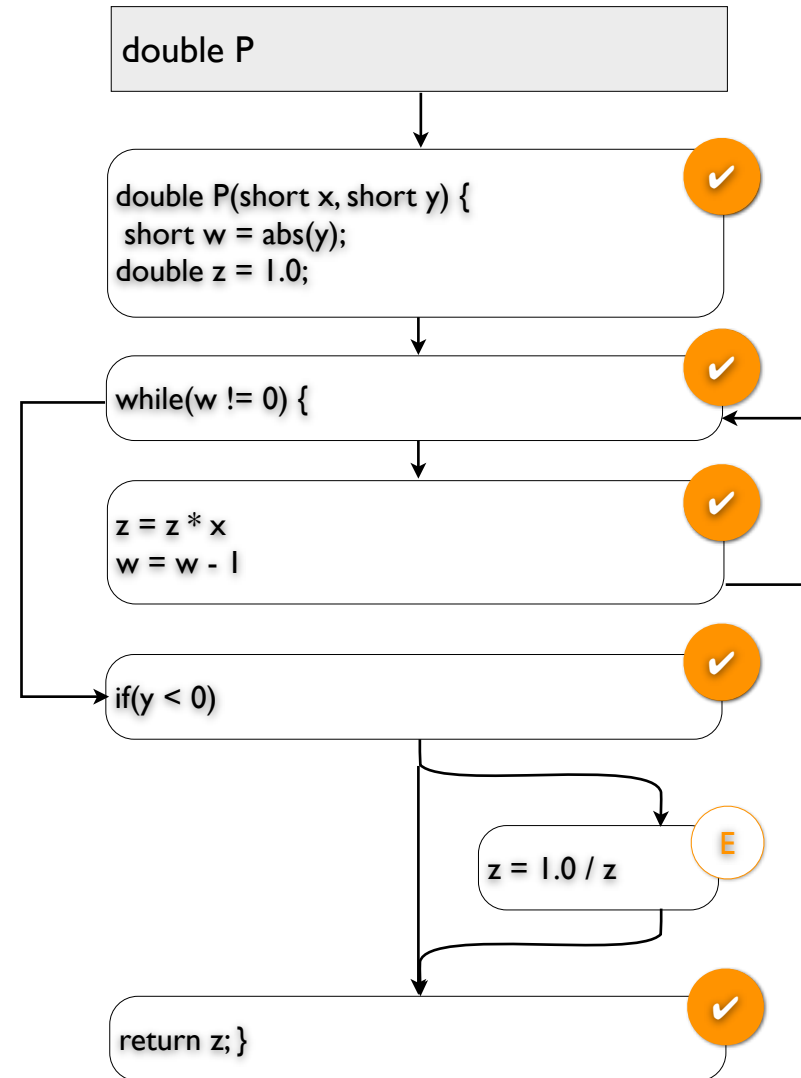
$\text{abs}(Y) \neq 0$

Satisfiable, e.g. $y=1$



Use the new concrete input to
execute the corresponding
feasible path

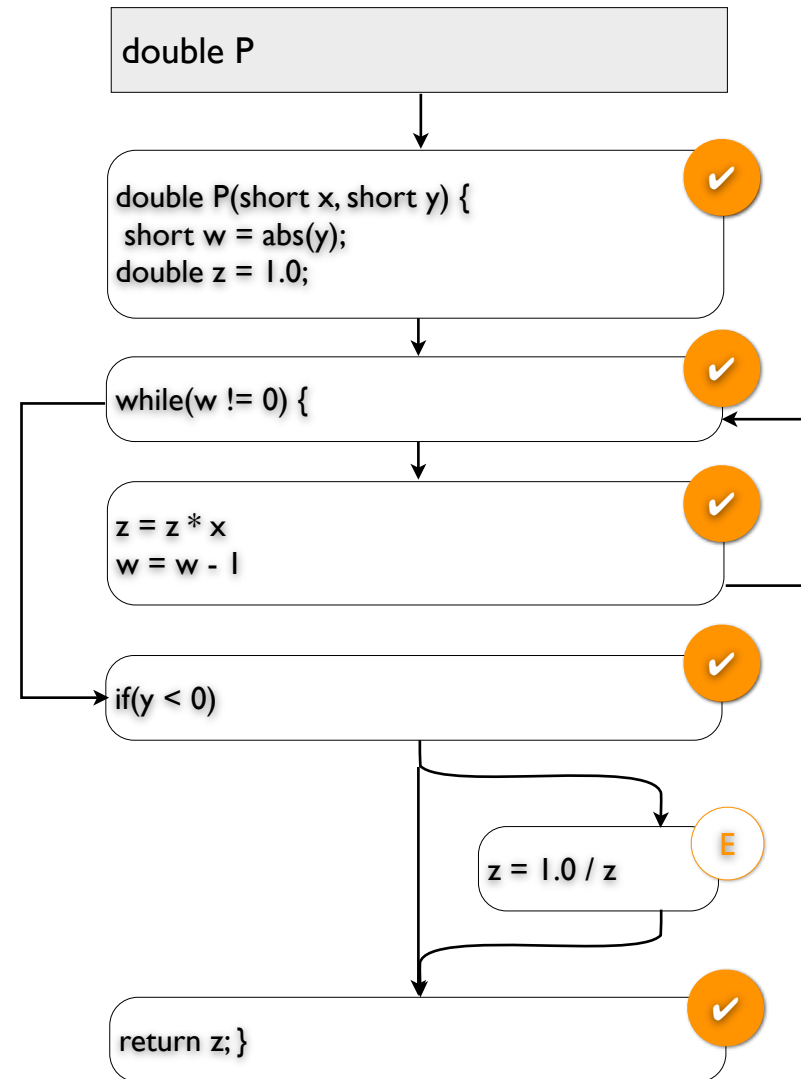
(0,1)



While executing, collect
constraints along path taken

$(0, 1)$

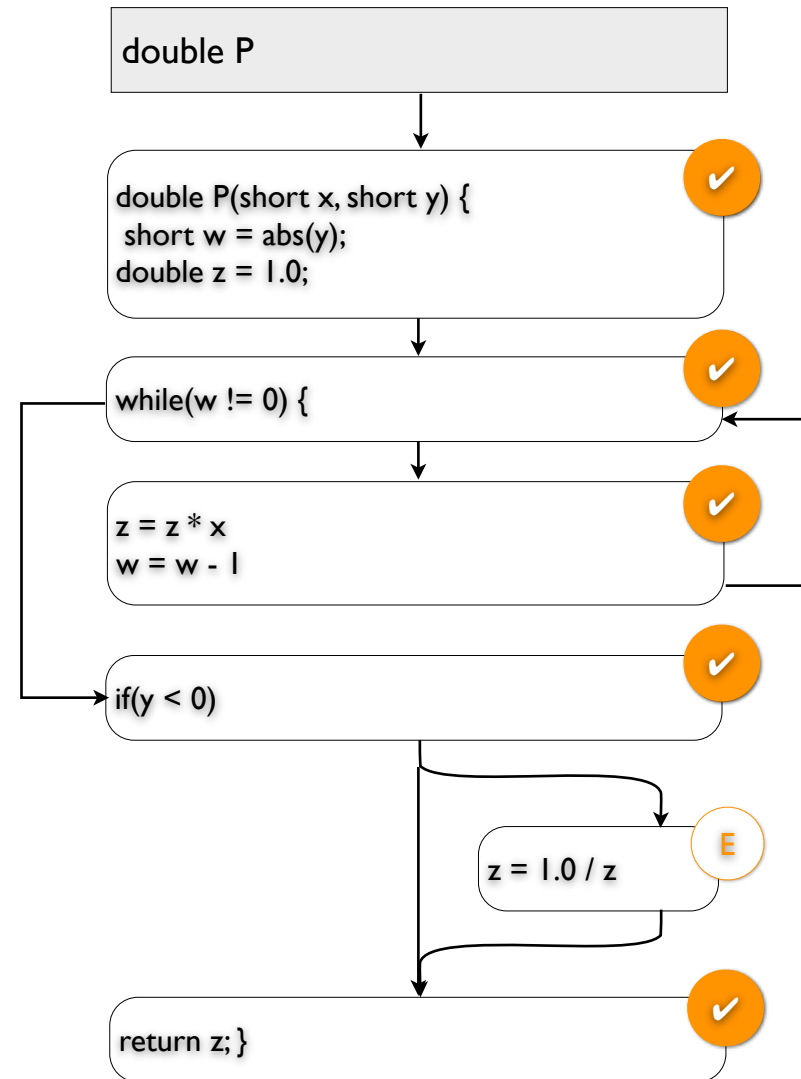
$\text{abs}(Y) \neq 0$
 $\text{abs}(Y) - 1 = 0$
 $\neg(Y < 0)$



Try to solve the CS where the last constraint is refuted

(0, 1)

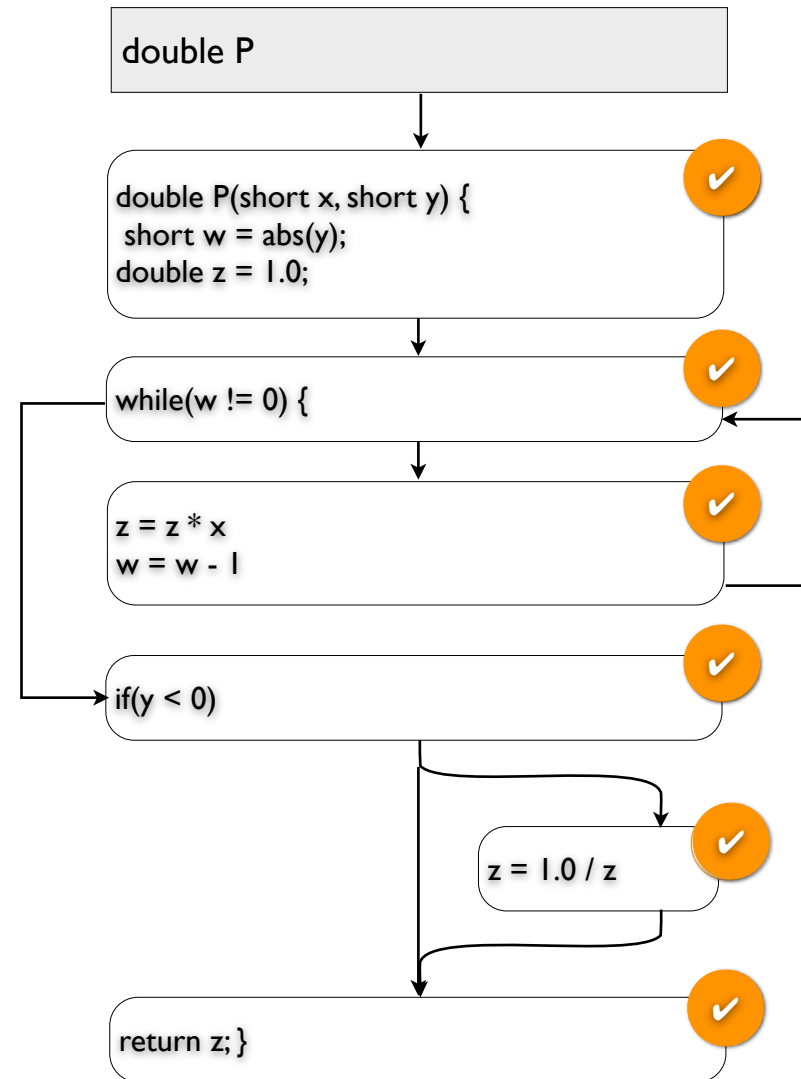
$\text{abs}(Y) \neq 0$
 $\text{abs}(Y) - 1 = 0$
 $(Y < 0)$



Repeat until all targets are covered

(0,-1)

$\text{abs}(Y) \neq 0$
 $\text{abs}(Y) - 1 = 0$
 $(Y < 0)$



Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$ generated by a random test-driver

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$ generated by a random test-driver

concrete $z = 9$

symbolic $z = x*x*x + 3*x*x + 9$

take then branch with constraint
 $x*x*x + 3*x*x + 9 \neq y$

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$ generated by a random test-driver

concrete $z = 9$

symbolic $z = x*x*x + 3*x*x + 9$

take then branch with constraint

$x*x*x + 3*x*x + 9 \neq y$

solve $x*x*x + 3*x*x + 9 = y$ to take else branch

Assume solver cannot handle this non-linear constraint

Stuck ?

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$ generated by a random test-driver

concrete $z = 9$

symbolic $z = x*x*x + 3*x*x + 9$

take then branch with constraint

$x*x*x + 3*x*x + 9 \neq y$

solve $x*x*x + 3*x*x + 9 = y$ to take else branch

Assume solver cannot handle this non-linear constraint

Stuck ?

Use concrete values!

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$
generated by a random test-driver

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$
generated by a random test-driver
concrete $z = 9$
symbolic $z = x*x*x + 3*x*x + 9$
cannot handle symbolic value of z

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$
generated by a random test-driver
concrete $z = 9$
symbolic $z = x*x*x + 3*x*x + 9$
cannot handle symbolic value of z
make symbolic $z = 9$ and proceed

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$
generated by a random test-driver
concrete $z = 9$
symbolic $z = x*x*x + 3*x*x + 9$
cannot handle symbolic value of z
make symbolic $z = 9$ and proceed
take then branch with constraint $9 \neq y$

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Let initially $x = -3$ and $y = 7$
generated by a random test-driver

concrete $z = 9$

symbolic $z = x*x*x + 3*x*x + 9$

cannot handle symbolic value of z

make symbolic $z = 9$ and proceed

take then branch with constraint $9 \neq y$

solve $9 = y$ to take else branch

execute next run with $x = -3$ and $y = 9$

got error (reaches abort)

Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Replace symbolic expression by concrete value when symbolic expression becomes unmanageable (i.e. non-linear)

Let initially $x = -3$ and $y = 7$
generated by a random test-driver

concrete $z = 9$

symbolic $z = x*x*x + 3*x*x + 9$

cannot handle symbolic value of z

make symbolic $z = 9$ and proceed

take then branch with constraint $9 \neq y$

solve $9 = y$ to take else branch

execute next run with $x = -3$ and $y = 9$

got error (reaches abort)

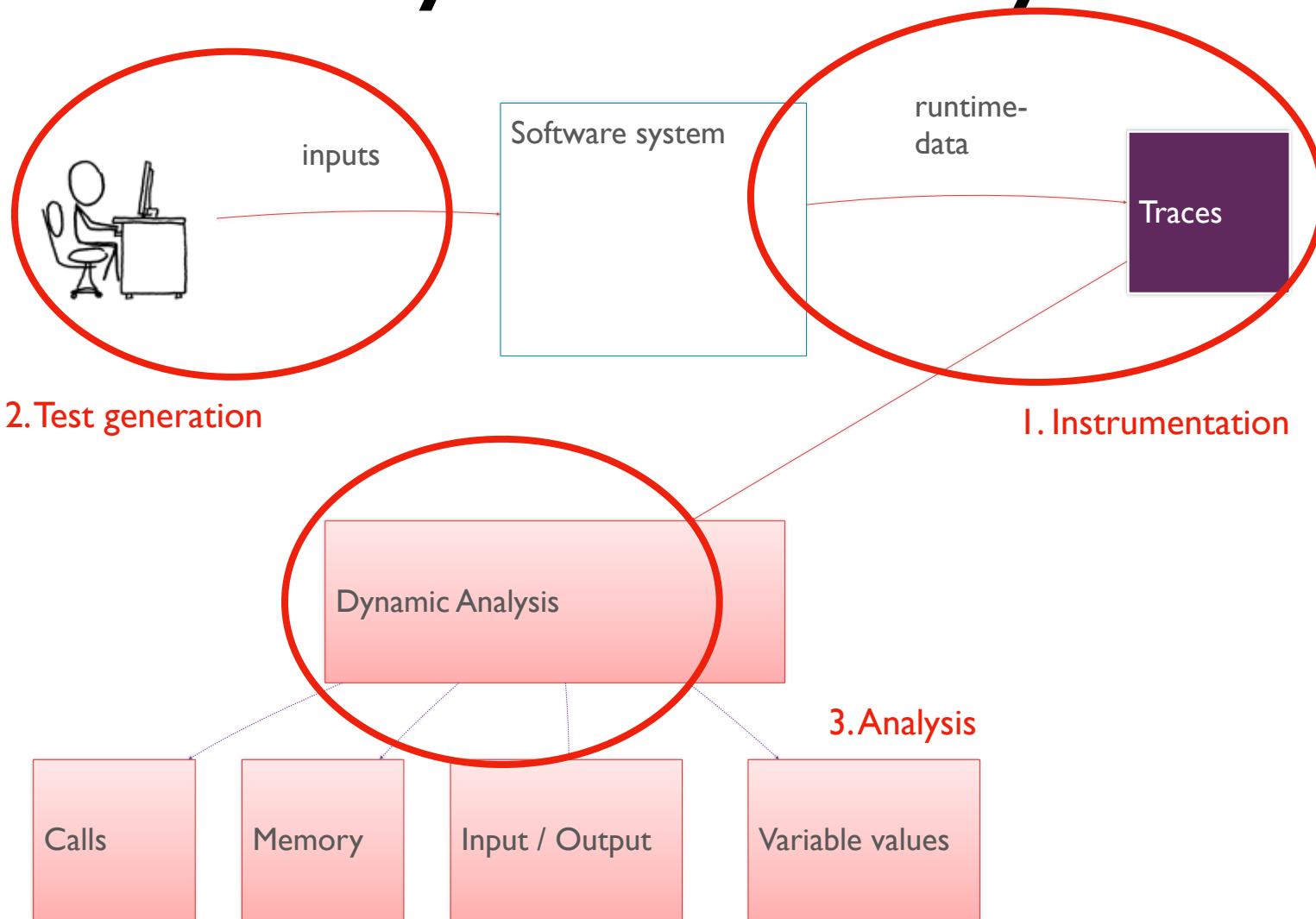
Simultaneous Symbolic & Concrete Execution

```
void test_me(int x, int y) {  
    z = x*x*x + 3*x*x + 9;  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

```
void test_me(int x, int y) {  
    z = black_box_fun(x);  
    if(z != y) {  
        printf("Good branch");  
    } else {  
        printf("Bad branch");  
        abort();  
    }  
}
```

Symbolic execution is only possible if the source code is available for instrumentation. This is often not the case, so "black box calls" are **executed concretely**.

Dynamic Analysis



Profiling

What's wrong with this? (I)

```
public class BuildBigString {  
    public final static int REPS = 80000;  
  
    // Builds/returns a big, important string.  
    public static String makeString() {  
        String str = "";  
        for (int n = 0; n < REPS; n++) {  
            str += "more";  
        }  
        return str;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(makeString());  
    }  
}
```

What's wrong with this? (2)

```
public class Fibonacci {
    public static void main(String[] args) {
        // print the first 10000 Fibonacci numbers
        for (int i = 1; i <= 10000; i++) {
            System.out.println(fib(i));
        }
    }

    // pre: n >= 1
    public static long fib(int n) {
        if (n <= 2) {
            return 1;
        } else {
            return fib(n - 2) + fib(n - 1);
        }
    }
}
```


What's wrong with this? (3)

```
public class WordDictionary {  
    // The set of words in our game.  
    List<String> words = new ArrayList<String>();  
  
    public void add(String word) {  
        words.add(word.toLowerCase());  
    }  
  
    public boolean contains(String word) {  
        for (String s : words) {  
            if (s.toLowerCase().equals(word)) {  
                return true;  
            }  
        }  
        return false;  
    }  
}
```

What's wrong with this? (4)

```
public class BankManager {
    public static void main(String[] args) {
        Account[] a = Account.getAll();
        for (int i = 0; i < Math.sqrt(895732); i++) {
            a[i].loadTaxData();
            if (a.meetsComplexTaxCode(2020)) {
                a[i].fileTaxes(4 * 4096 * 17);
            }
        }

        Account[] a2 = Account.getAll();
        for (int i = 0; i < Math.sqrt(895732); i++) {
            if (a.meetsComplexTaxCode(2020)) {
                a2[i].setTaxRule(4 * 4096 * 17);
                a2[i].save(new File(a2.getName()));
            }
        }
    }
}
```

The correct answers

1. Who cares?

2. Who cares?

3. Who cares?

4. Who cares?

- "We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil."
-- Donald Knuth

- "We follow two rules in the matter of optimization:
 1. Don't do it.
 2. (for experts only) Don't do it yet."

-- M.A. Jackson

Optimisation Metrics

- Runtime / CPU usage
 - what lines of code the program is spending the most time in
 - what call/invocation paths were used to get to these lines
- Memory usage
 - what kinds of objects are on the heap
 - where were they allocated
 - who is pointing to them now
 - "memory leaks"

Types of Profiling

- **Insertion:** placing special profiling code into your program (manually or automatically)
 - pros: can be used across a variety of platforms; accurate
 - cons: may require recompiling; profiling code may affect performance
- **Sampling:** monitoring CPU or VM at regular intervals and saving a snapshot of CPU and/or memory state
 - pros: no modification of app is necessary
 - cons: less accurate; varying sample interval leads to a time/accuracy trade-off; small methods may be missed; cannot easily monitor memory usage

Tools

- Many Java profiling/optimisation tools available:
 - VisualVM
 - YourKit
 - Java Flight Recorder (IntelliJ)
 - Async Profiler (IntelliJ)
 - Java Memory Profiler (JMP)
 - Eclipse Memory Analyzer
- hprof (java -Xrunhprof)

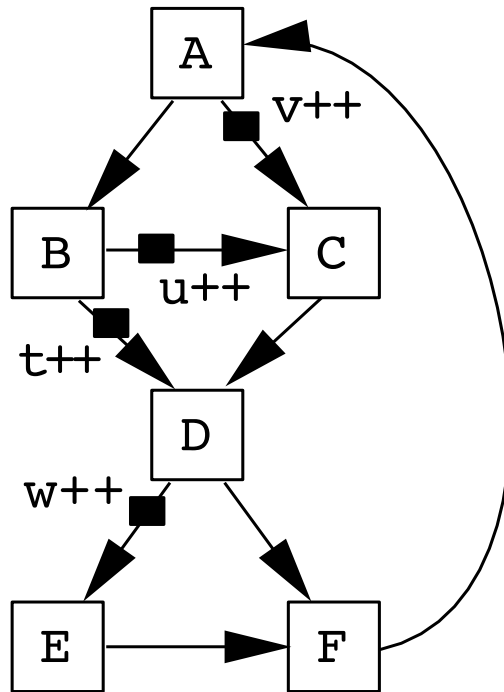
Path Profiling

- Goal: Count how often a path through a function is executed
- Interesting for various applications
- Profile-directed compiler optimisations
- Performance tuning: Which paths are worth optimising?
- Test coverage: Which paths are not yet tested?

Edge Profiling

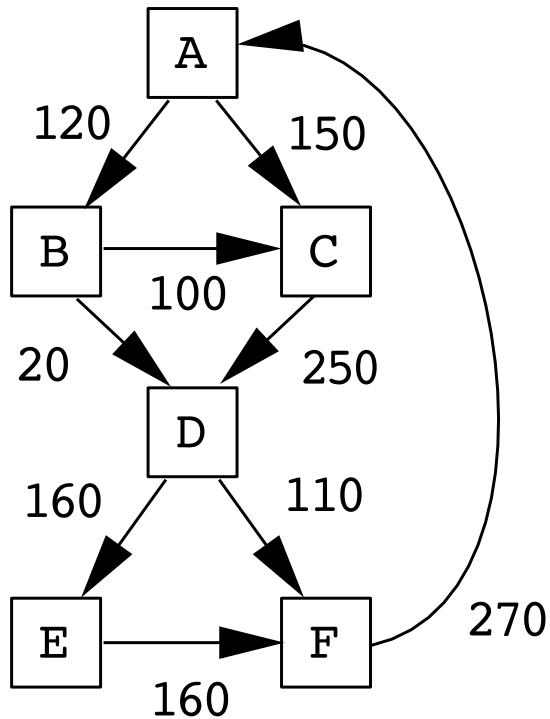
- Naive approach: Edge profiling
- Instrument each branching point
- Count how often each CFG edge is executed
- Estimate most frequent path: Always follow most frequent edge

Edge Profiling



$C \rightarrow D$	$=$	$u + v$
$D \rightarrow F$	$=$	$t + u + v - w$
$E \rightarrow F$	$=$	w
$A \rightarrow B$	$=$	$t + u$
$F \rightarrow A$	$=$	$t + u + v$

Path Profiling



Path	Prof1	Prof2
ACDF	90	110
ACDEF	60	40
ABCDF	0	0
ABCDEF	100	100
ABDF	20	0
ABDEF	0	20

What is the most frequent path?

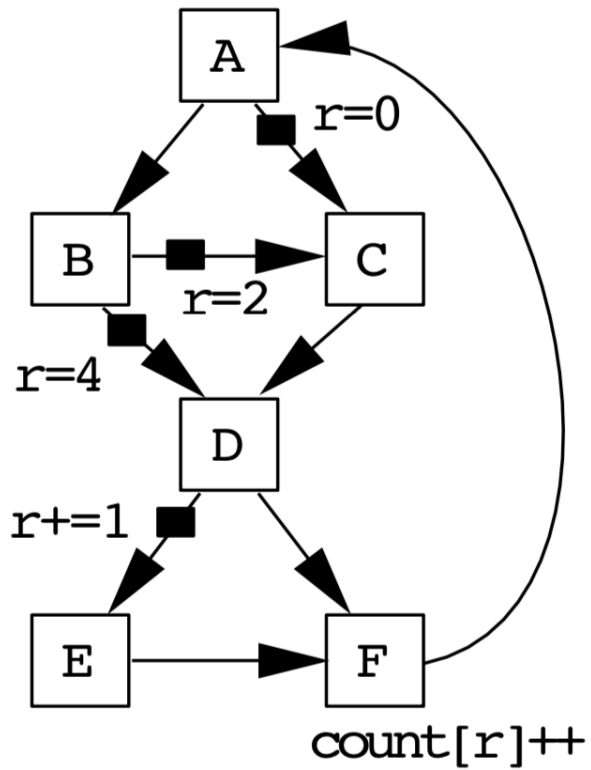
Path Profiling

- Naive approach: Use edge profiling
- Instrument each branching point
- Count how often each CFG edge is executed
- Estimate most frequent path: Always follow most frequent edge
- *Fails to uniquely identify most frequent path*

Ball-Larus Algorithm

- Thomas Ball and James R. Larus. "Efficient path profiling." Proceedings of the 29th Annual ACM/IEEE International Symposium on Microarchitecture. IEEE Computer Society, 1996.
- Assign a number to each path
- Compute path number by incrementing a counter at branching points
- Properties of path encoding
 - Precise: A single unique encoding for each path
 - Minimal: Instruments subset of edges with minimal cost

Path Profiling



Path	Encoding
------	----------

ACDF	0
------	---

ACDEF	1
-------	---

ABCDF	2
-------	---

ABCDEF	3
--------	---

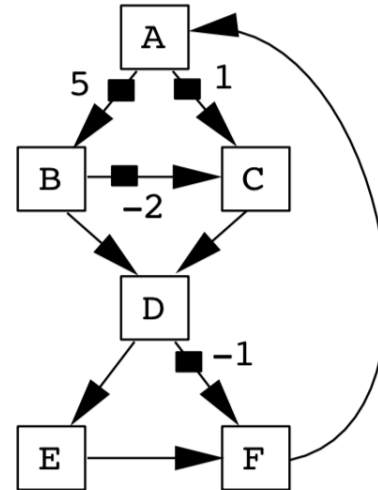
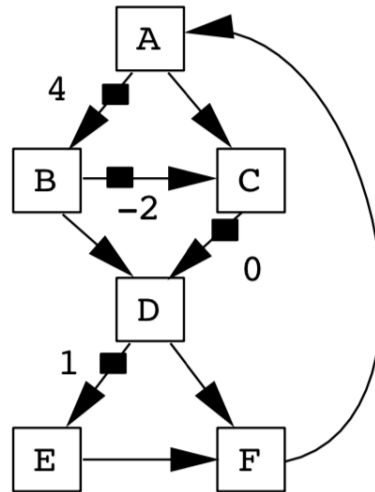
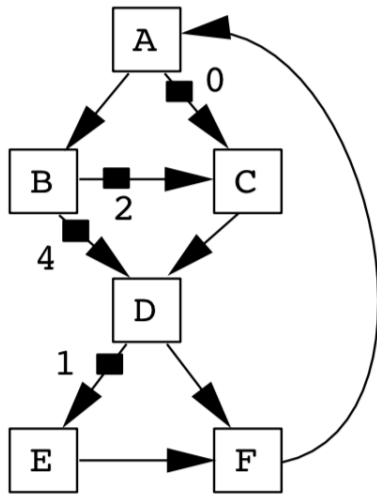
ABDF	4
------	---

ABDEF	5
-------	---

Assumptions

- Control flow graph is a directed acyclic graph (DAG)
- n paths (numbered 0 to $n-1$)
- Graph has unique entry and exit nodes
- Artificial back edge from exit to entry

Instrumentation Options



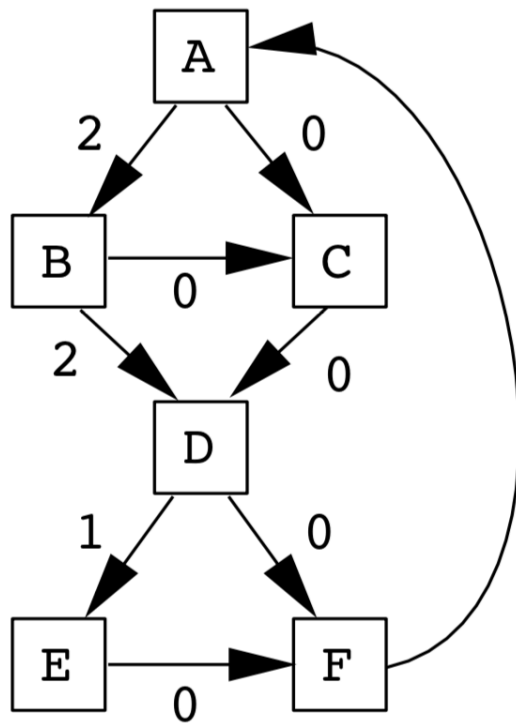
Algorithm: Overview

- Step 1:Assign integers to edges
 - Goal: Sum along a path yields unique number for path
 - Enough to achieve "precise" goal
- Step 2:Assign increment operations to edges
 - Goal: Minimise additions along edges
 - Instrument subset of all edges
 - Assumes to know/estimate how frequent edges are executed

Representing Paths with Sums

- Associate with each node a value:
 $\text{NumPaths}(n)$ = number of paths from n to exit
- Computing NumPaths
 - Visit nodes in reverse topological order
 - If n is leaf node:
 $\text{NumPaths}(n) = 1$
 - Else:
 $\text{NumPaths}(n)$ = sum of NumPaths of destination of outgoing edges

Example



Vertex v	NumPaths(v)
A	6
B	4
C	2
D	2
E	1
F	1

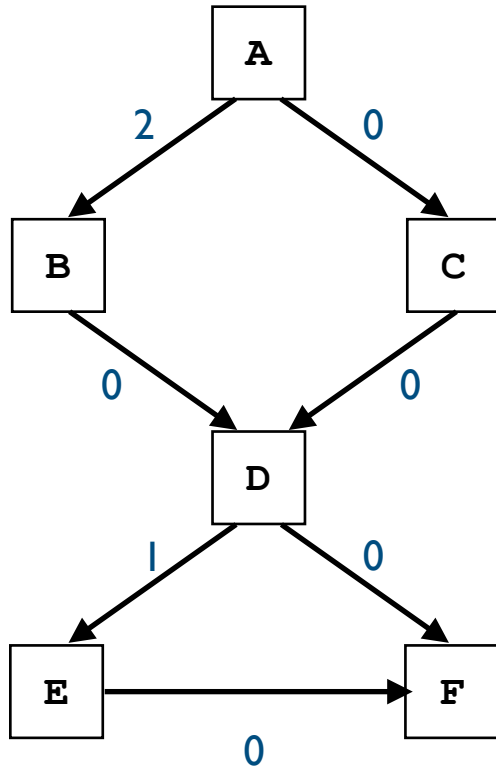
Assigning Values

```
foreach vertex v in reverse topological order {  
    if v is a leaf vertex {  
        NumPaths(v) = 1;  
    } else {  
        NumPaths(v) = 0;  
        for each edge e = v->w {  
            NumPaths(v) = NumPaths(v) + NumPaths(w);  
        }  
    }  
}
```

Assigning Values

```
foreach vertex v in reverse topological order {  
  if v is a leaf vertex {  
    NumPaths(v) = 1;  
  } else {  
    NumPaths(v) = 0;  
    for each edge e = v->w {  
      Val(e) = NumPaths(v);  
      NumPaths(v) = NumPaths(v) + NumPaths(w);  
    }  
  }  
}
```

Example



Node n	NumPath (n)
F	1
E	1
D	2
C	2
B	2
A	4

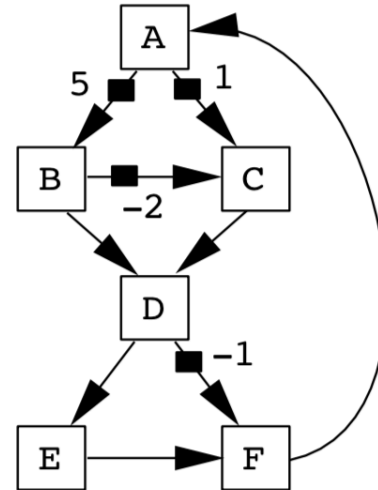
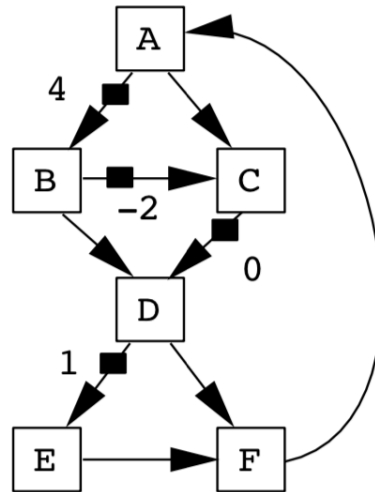
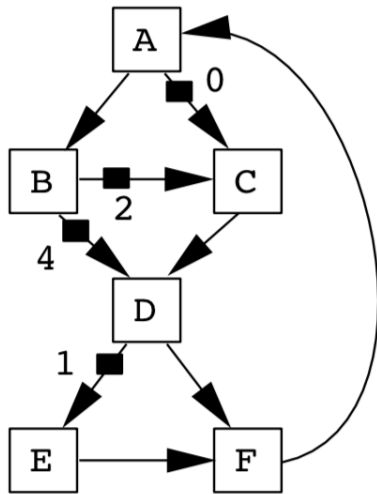
Algorithm: Overview

- Step 1: Assign integers to edges
 - Goal: Sum along a path yields unique number for path
 - Enough to achieve "precise" goal
- Step 2: Assign increment operations to edges
 - Goal: Minimise additions along edges
 - Instrument subset of all edges
 - Assumes to know/estimate how frequent edges are executed

Spanning Tree

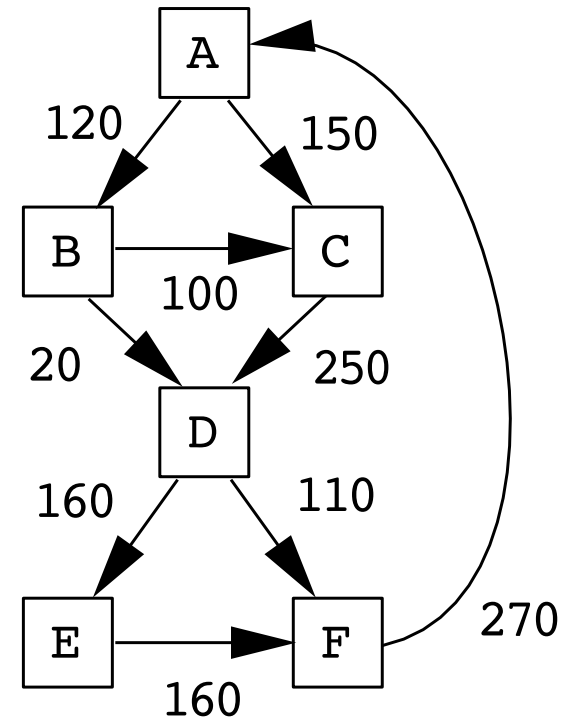
- Given: Graph G
- Spanning tree T :
Undirected subgraph of G that is a tree and that contains all nodes of G
- Chord edges: Edges in G but not in T

Spanning Trees

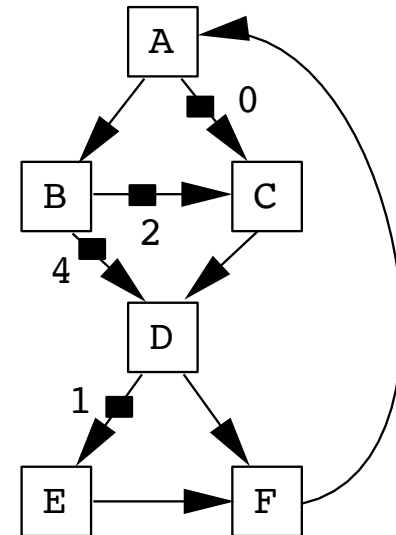
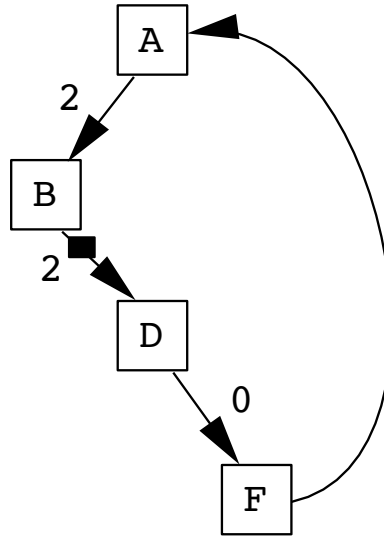
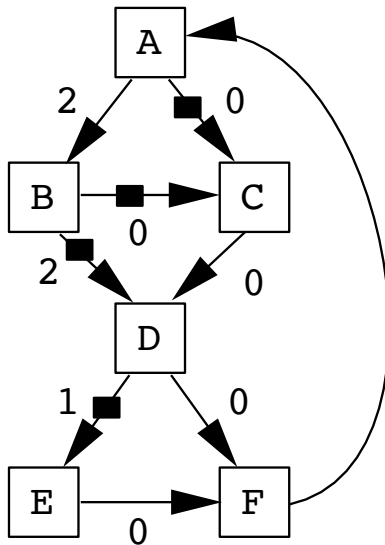


Increments for Edges

- Goal: Increment sum at subset of edges
- Choose spanning tree with maximum edge cost
 - Cost of individual edges is assumed to be known
- Compute increments at the chords of the spanning tree
 - Based on existing event counting algorithm



Determining Chord Increments

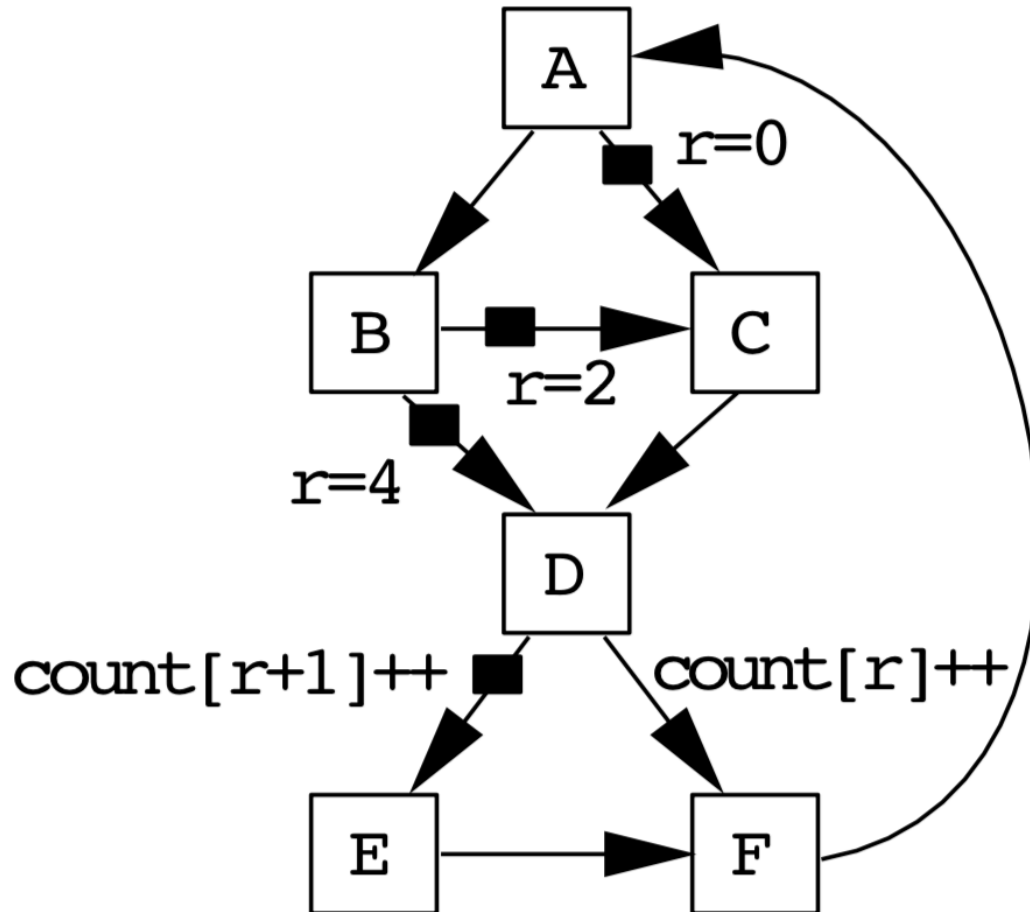


$$\begin{aligned}
 \text{Inc}(B \rightarrow D) &= \text{Val}(A \rightarrow B) + \text{Val}(B \rightarrow D) + \text{Val}(D \rightarrow F) + \text{Val}(F \rightarrow A) \\
 &= 2 + 2 + 0 + 0 \\
 &= 4
 \end{aligned}$$

Instrumentation

- Basic idea
 - Initialise sum at entry: $r=0$
 - Increment at edges: $r+=..$
 - At exit, increment counter for path:
 $\text{count}[r]++$
- Optimisation
 - Initialise with incremented value, if first chord on path: $r=..$
 - Increment sum and counter for path, if last chord on path: $\text{count}[r+..]+$
+

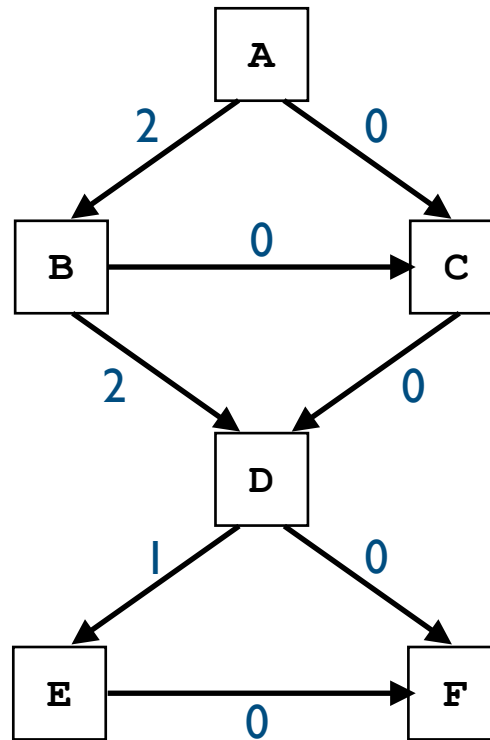
Optimisation



Regenerating the Path

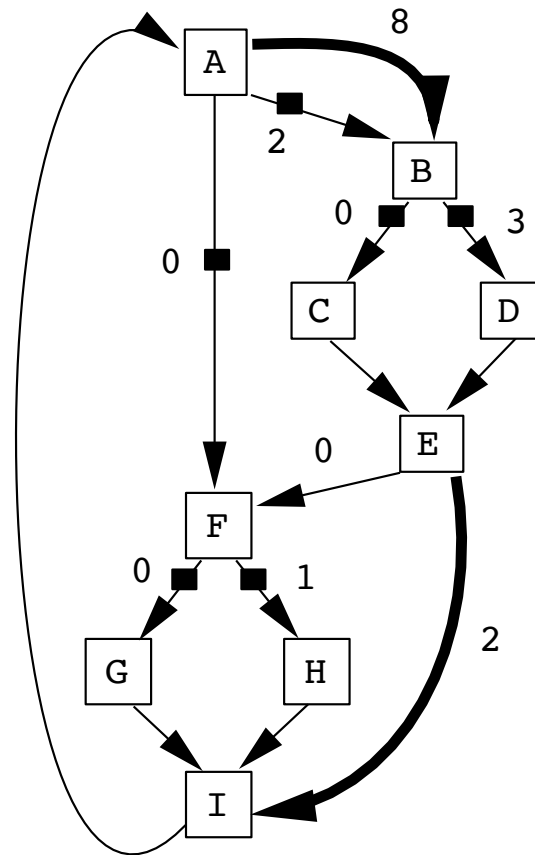
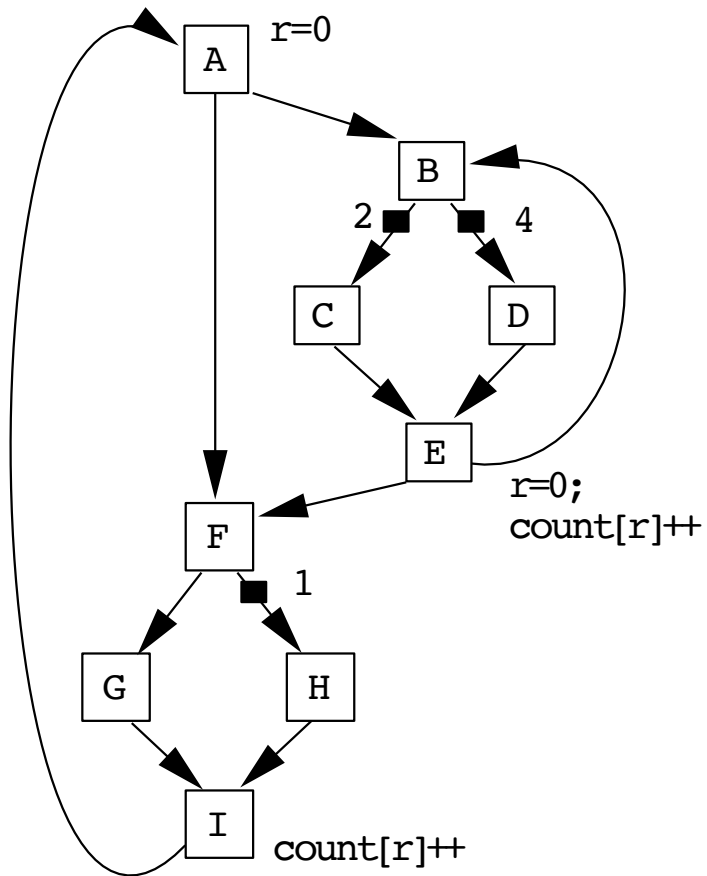
- Knowing the sum r , how to determine the path?
- Use edge values from step 1 ("non-minimal increments")
- Start at entry with $R = r$
- At branches, use edge with largest value v that is smaller than R and set $R \leftarrow v$

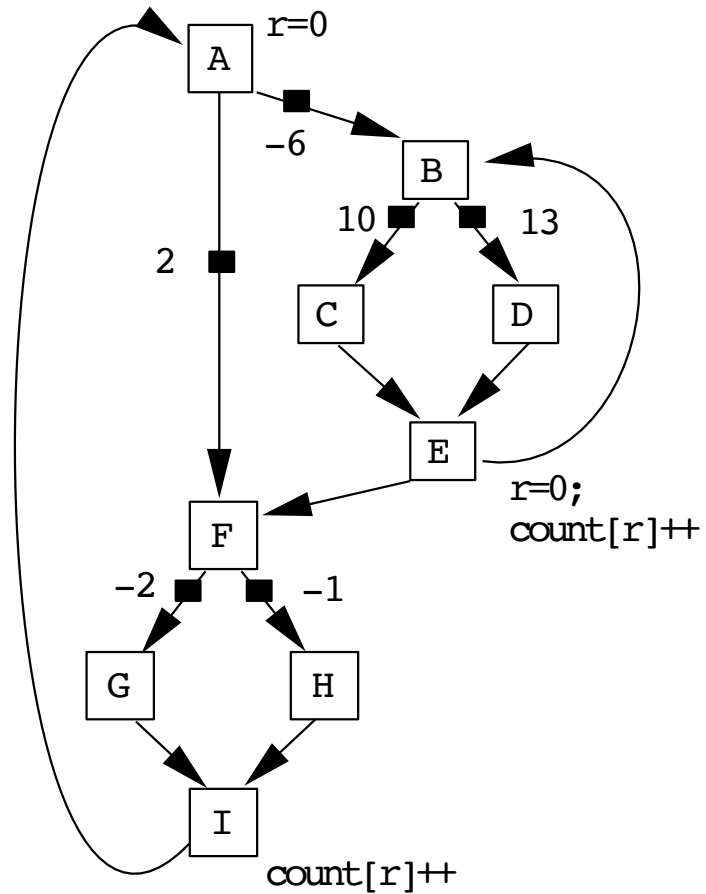
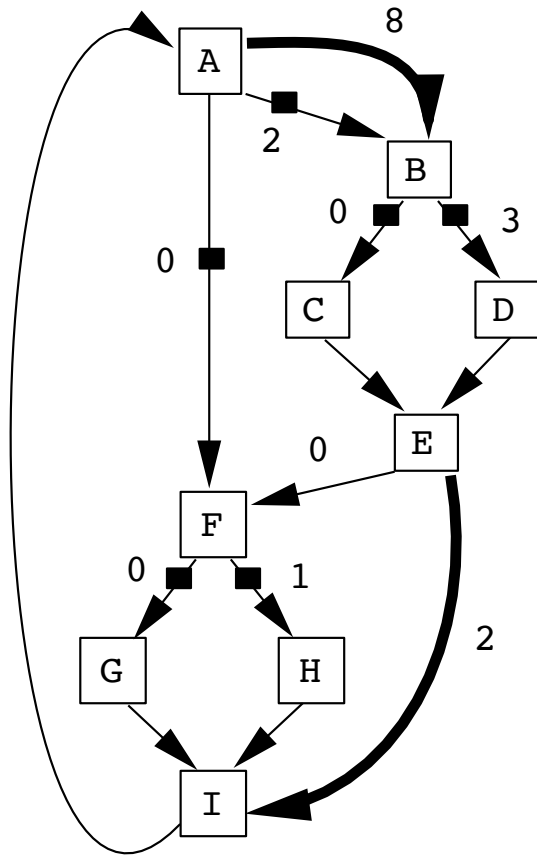
Regenerating the Path



Generalising to Cyclic CFGs

- For each backedge $n \rightarrow m$, add dummy edges
 - $\text{Entry} \rightarrow m$
 - $n \rightarrow \text{Exit}$
- Remove backedges and add DAG-based increments
- In addition, add instrumentation to each backedge
 - `count[r]++; r=0`





Path	Path Sum
AFGI	0
AFHI	1
ABCEFGI	2
ABCEFHI	3
ABDEFGI	5
ABDEFHI	6
ABCE	4
ABDE	7
BCE	10
BDE	13
BCEFGI	8
BCEFHI	9
BDEFGI	11
BDEFHI	12